

Current Status

Current Application Status

Backend

- ☒ GET All Todos
- ☒ Dynamic Search
- ☒ Dynamic Filtering
- ☒ Dynamic Sorting
- ☒ POST Create Todo
- ☒ PostgreSQL Integration

Frontend

- ☒ Fetch Todos from Backend
- ☒ Create Todo
- ☒ Search
- ☒ Filter
- ☒ Sorting
- ☒ Dynamic Query Parameters
- ☒ Automatic Data Refresh

Frontend Pending Tasks

Frontend (React) Pending Tasks

CRUD Integration

- ☒ Load todos from PostgreSQL on application start.
 - ☒ Create Todo using `POST /todos`.
 - ☐ Update Todo using `PUT /todos/:id`.
 - ☐ Delete Todo using `DELETE /todos/:id`.
 - ☐ Mark Todo as Completed/Active using API instead of local state.
-

Search & Filtering

- ☒ Filter by Status.
 - ☒ Filter by Priority.
 - ☒ Search by Title and Details (client-side).
 - ☒ Move search to backend.
 - ☒ Move filtering to backend.
-

Sorting

- ☒ Sort by Title.
 - ☒ Sort by Created ID.
 - ☒ Move sorting to backend.
-

Pagination

- ☒ Client-side pagination.
 - ☐ Backend pagination.
 - ☐ Page size selector (10/20/50).
-

Todo Form

- ☒ ~~Title.~~
 - ☒ ~~Details.~~
 - ☒ ~~Due Date.~~
 - ☒ ~~Priority.~~
 - ☐ Validate due date:
 - ☐ New todo → today or future only.
 - ☐ Update → existing due date or future.
 - ☐ Character counter for Title.
 - ☐ Character counter for Details.
 - ☐ Disable Add button while API request is in progress.
-

AI Features

- ☒ ~~Improve Todo.~~
 - ☒ ~~Editable AI suggestion.~~
 - ☒ ~~Accept / Cancel suggestion.~~
 - ☒ ~~Generic prompt builder.~~
 - ☐ Loading spinner animation.
 - ☐ Keyboard shortcut (Enter to Accept suggestion).
 - ☐ AI Summary of Todo.
 - ☐ AI Generate Title from Details.
 - ☐ AI Improve Description.
 - ☐ Prompt templates for additional contexts.
-

Voice Input

- ☒ ~~Speech to text.~~
 - ☐ Voice input for Details field.
 - ☐ Stop recording on silence.
 - ☐ Display listening animation.
-

UI/UX

- ☒ ~~Tailwind CSS integration.~~
 - ☒ ~~Responsive input layout.~~
 - ☒ ~~Priority badge with colors.~~
 - ☒ ~~Due date display (~~dd-mm-yyyy~~).~~
 - ☐ Toast notifications.
 - ☐ Loading skeleton while fetching todos.
 - ☐ Empty state illustration.
 - ☐ Confirmation dialog before Delete.
 - ☐ Dark Mode.
 - ☐ Accessibility improvements (ARIA labels, keyboard navigation).
-

Error Handling

- ☒ ~~Friendly AI error messages.~~
- ☐ Global API error handling.
- ☐ Offline/Network error message.
- ☐ Retry button for failed API requests.

Suggested Development Order

Based on your current progress, I'd recommend this sequence:

1. Replace all frontend local state mutations with API calls - **WORKING**.
2. React component tests.
3. Add advanced AI features and other enhancements.

Backend Pending Tasks

Backend (Node.js + Express) Pending Tasks

CRUD APIs

- ☐ GET `/todos`
 - ☒ ~~GET `/todos/:id`~~
 - ☒ ~~POST `/todos`~~
 - ☐ PUT `/todos/:id`
 - ☐ DELETE `/todos/:id`
-

Validation

- ☐ Required field validation.
 - ☐ UUID validation.
 - ☐ Due date validation.
 - ☐ Priority validation.
 - ☐ Input sanitization.
-

Search & Filters

- ☒ ~~Search by title/details.~~
 - ☒ ~~Filter by priority.~~
 - ☒ ~~Filter by status.~~
 - ☒ ~~Sort by title.~~
 - ☒ ~~Sort by created date.~~
 - ☐ Pagination using `LIMIT` and `OFFSET`.
-

Error Handling

- ☐ Global Express error handler.
 - ☐ Custom error classes.
 - ☐ Standard API response format.
 - ☐ HTTP status code consistency.
-

Environment Configuration

- ☐ `.env` support.
 - ☐ Move DB credentials to environment variables.
 - ☐ Move frontend URL to environment variables.
 - ☐ Add `.env.example`.
-

Authentication

- ☐ User Registration.
 - ☐ Login.
 - ☐ Password hashing.
 - ☐ JWT Authentication.
 - ☐ Protected routes.
 - ☐ User-specific todos.
-

Database

- ☒ PostgreSQL connection.
- ☒ `user_todos` table.
- ☒ Add `status` column (currently filtering assumes it exists).
- ☐ Automatic `updated_at` update on modification.
- ☐ Seed script.

Testing

- ☐ React component tests.
- ☐ API unit tests.
- ☐ Repository tests.
- ☐ Integration tests.
- ☐ End-to-end tests.

CI/CD

- ☐ GitHub Actions.
- ☐ Linting.
- ☐ Automated testing.
- ☐ Build pipeline.
- ☐ Backend deployment.
- ☐ Frontend deployment.

Future Enhancements

- ☐ File attachments.
- ☐ Reminder notifications.
- ☐ Recurring todos.
- ☐ Todo categories/tags.
- ☐ Dashboard with statistics.
- ☐ AI task prioritization.
- ☐ AI due date suggestions.
- ☐ Export to PDF/CSV.
- ☐ Calendar view.
- ☐ Drag-and-drop ordering.
- ☐ Real-time updates (WebSockets).

Suggested Development Order

Based on your current progress, I'd recommend this sequence:

1.  Complete backend CRUD (**POST**, **PUT**, **DELETE**).
2. Move search, filtering, sorting, and pagination to the backend - **WORKING**.
3. Add backend validation and centralized error handling.
4. Write backend API tests.
5. Add JWT authentication and user-specific todos.
6. Set up CI/CD and deploy the application.

FrontEnd Daily status

F 27-07-2026

1. POST API Integration

- Integrated the Create Todo API with the React application.
- Converted the todo form into JSON using `JSON.stringify(formData)` before sending it to the backend.
- Successfully saved:
 - Title
 - Details
 - Due Date
 - Priority

2. Understanding API Calls in React

- Learned that API calls triggered directly by user actions (such as clicking **Add Todo**) belong inside event handlers.
- Understood that only render-driven synchronization belongs inside `useEffect`.

3. Server-side Filtering

Integrated backend filtering with the UI for:

- Search by Title/Details
- Filter by Priority
- Filter by Status
- Sort by Title (A-Z / Z-A)
- Sort by Created Date (Newest / Oldest)

4. Improved Data Fetching

- Replaced multiple API calls scattered across different event handlers with a single `useEffect`.
- Configured the effect to automatically reload data whenever the following state changes:
 - Search text
 - Priority filter
 - Status filter
 - Sort field
 - Sort order

This created a cleaner and more maintainable data-fetching flow.

5. Query String Helper

- Created a reusable helper function to generate query strings using `URLSearchParams`.

- Configured it to send only meaningful parameters, omitting empty search values and default filters.

6. Request Configuration

- Standardized request headers across API calls.
 - Configured requests to send JSON payloads using the `Content-Type: application/json` header.
-

Key Concepts Learned Today

- Difference between render-driven side effects (`useEffect`) and event-driven API calls.
- Clean React architecture by centralizing data fetching.
- Building reusable query parameters with `URLSearchParams`.
- End-to-end integration between React, Express, and PostgreSQL.

BackEnd Daily Status

B 27-07-2026

1. Query Parameters

- Learned to retrieve query parameters from the request using `req.query`.
- Implemented dynamic filtering based on the values received from the client.

2. Dynamic SQL for GET Requests

- Built dynamic SQL queries by constructing `WHERE` conditions only for the filters supplied by the client.
- Used parameterized SQL placeholders (`$1`, `$2`, ...) to safely execute queries and prevent SQL injection.
- Implemented search, filtering, and sorting using dynamically generated SQL.

3. POST API Development

- Processed request payloads using `req.body`.
- Generated dynamic `INSERT` statements based on the incoming object properties.
- Created parameterized placeholders dynamically for each column.
- Used PostgreSQL's `RETURNING *` clause to immediately return the inserted record without executing an additional `SELECT` query.

Key Concepts Learned Today

- Dynamic SQL query construction for filtering and inserting data.
- Safe parameterized SQL using placeholders.
- PostgreSQL `RETURNING *` for retrieving inserted records.
- End-to-end integration between React, Express, and PostgreSQL.